

Relational Database Concept Reference Card

Relational Database Concept Reference Card

Supporting Chapters 2 and 12 — The Science of Learning SQL

Core Building Blocks (Chapter 2)

A relational database stores data in separate, interconnected tables rather than one flat spreadsheet. Think of it as a filing cabinet where each drawer holds a focused set of records, and labelled tabs connect related drawers together.

Concept	Definition	Quick Example
Table	A named collection of rows and columns representing one subject	customers, orders, products
Row (Record)	One complete, individual unit of data	A single customer's full details
Column (Field)	A named attribute enforcing a specific data type	email VARCHAR, order_date DATE
Primary Key (PK)	A column (or combination) that uniquely identifies every row	customer_id INT — no duplicates allowed
Foreign Key (FK)	A column that references a primary key in another table	orders.customer_id → customers.customer_id
Data Type	The rule that controls what kind of value a column stores	INT, VARCHAR, DATE, DECIMAL, BOOLEAN

Common SQL Data Types

- INT / INTEGER — Whole numbers; ideal for IDs and counts
- VARCHAR(n) — Variable-length text up to n characters; names, emails, descriptions
- DATE / DATETIME — Calendar dates and timestamps; transaction records, event logs
- DECIMAL(p, s) — Precise numbers with decimal places; prices, financial figures
- BOOLEAN — True/false flags; active status, feature toggles
- TEXT — Unrestricted-length strings; notes, free-form comments

Rule of thumb: Each column enforces exactly one data type. Storing "2024-01-15" in a VARCHAR column instead of a DATE column breaks sorting, filtering, and date arithmetic.

Keys and Relationships

- A primary key gives every row a permanent, reliable address — the database guarantees no two rows share the same value.
- A foreign key creates the link between tables, making cross-table JOIN queries possible and meaningful.
- A foreign key value must either match an existing primary key in the referenced table or be NULL — this is referential integrity.

```
customers      orders
-----
customer_id (PK) ← customer_id (FK)
name           order_id (PK)
email          order_date
               total_amount
```

Schema Vocabulary (Chapter 12)

A schema is the blueprint of a database — the full map of tables, columns, data types, primary keys, and foreign keys that defines every JOIN you will ever write.

Term	What It Means in Practice
Schema	Complete structural blueprint of the database
Normalisation	Splitting data into focused tables to eliminate redundancy and prevent update anomalies
1NF / 2NF / 3NF	The three normal forms — each level removes a specific category of redundancy
Fact Table	Central table in a warehouse schema storing measurable events (sales, clicks, transactions)
Dimension Table	Descriptive table providing context for facts (customers, products, dates, regions)
Star Schema	Fact table surrounded directly by dimension tables — simpler joins, faster queries
Snowflake Schema	Dimension tables further normalised into sub-dimensions — more joins required
Index	A data structure that lets the database locate rows without scanning the entire table
Data Dictionary	Documentation describing every table, column, data type, and key in a production database

Normalisation at a Glance

- First Normal Form (1NF): Every column holds one atomic value — no comma-separated lists inside a single cell.

- Second Normal Form (2NF): Every non-key column depends on the whole primary key, not just part of it.
- Third Normal Form (3NF): Every non-key column depends only on the primary key — no column describes another non-key column.

Why it matters for analysts: Normalisation prevents update anomalies — the silent errors that corrupt analytical results when the same fact is stored in multiple places and updated inconsistently.

Star vs. Snowflake: Quick Decision Guide

- Star schema → fewer joins, easier to read, preferred for ad-hoc reporting and dashboards.
- Snowflake schema → more joins required, but dimension data is fully normalised and storage-efficient.
- Recognising which structure you are querying determines how many JOIN clauses your analytical reports will need.

Indexes: The Performance Lever

- An index on a frequently filtered or joined column can reduce query time from minutes to seconds.
- Missing indexes on foreign key columns are the single most common cause of slow JOIN performance.
- Read production data dictionaries and schema documentation to identify which indexes already exist before optimising a query.

Keep this card beside you when reading Chapters 2 and 12, and revisit it every time you write a JOIN or design a new query against an unfamiliar schema.